

Secure File Transfer Monitoring and Intrusion Detection System

Submitted By	Nirmiti Patil
Role	Cybersecurity Intern
Organization	Unified Mentor

ABSTRACT:

SECUREXFER v3.0 is a secure file transfer and monitoring system developed using Python and Streamlit. The project provides encrypted file transfer, integrity verification, user authentication, alert monitoring, and audit reporting in a modern web-based interface. The system follows a 6-step workflow: Monitor, Classify, Hash, Authorize, Alert, and Report. It ensures that transferred files remain confidential, authenticated, and tamper-free during transmission. SQLite database logging, SHA-256 hashing, AES-based encryption using Fernet, and reporting modules are integrated to improve security and monitoring capabilities.

INTRODUCTION:

File transfer systems are widely used in organizations for sharing sensitive data. Traditional transfer methods may expose files to unauthorized access, tampering, or data leakage. SECUREXFER v3.0 is designed to solve these issues by implementing secure encryption, integrity validation, classification of files, event logging, and automated reporting. The project provides a user-friendly interface where users can securely upload, encrypt, transfer, monitor, and download files while maintaining complete audit logs and security alerts.

OBJECTIVES:

The main objectives of the project are:

- To provide secure encrypted file transfer.
- To verify file integrity using SHA-256 hashing.
- To classify sensitive and non-sensitive files.
- To monitor suspicious activities and generate alerts.
- To maintain audit logs using SQLite database.
- To generate TXT, CSV, and JSON security reports.

- To provide an interactive web dashboard for monitoring.

TECHNOLOGIES USED:

Technology	Purpose
Python	Backend Development
Streamlit	Web Interface
SQLite	Database Storage
Cryptography Library	Encryption & Decryption
Pandas	Data Processing
Plotly	Data Visualization
Hashlib	SHA-256 Hashing
Logging Module	System Logging

SYSTEM WORKFLOW:

Step No.	Workflow Stage	Description	Technologies/ Methods Used
1	Monitor	The system monitors file transfer activities and records all filesystem events into the SQLite database for auditing and tracking.	SQLite, Logging Module
2	Classify	Uploaded files are analyzed based on file extensions and sensitive keywords to identify confidential or risky files.	Python Rules Engine, Keyword Matching
3	Hash	SHA-256 hashing is performed before and after transfer to verify file integrity and detect tampering.	Hashlib (SHA-256)
4	Authorize	The system validates whether the file type and operation are authorized according to predefined security rules.	Authorization Rules, Extension Validation
5	Alert	Suspicious or unauthorized activities trigger alerts and are displayed in the alerts dashboard while being stored in logs.	Logging, Alert System, SQLite
6	Report	The system generates detailed audit and compliance reports in TXT, CSV, and JSON formats with visual analytics.	Pandas, Plotly, Report Generator

FEATURES:

Security Features

- AES-based encryption using Fernet
- Password-based key derivation (PBKDF2-SHA256)
- SHA-256 integrity verification
- Sensitive file detection
- Unauthorized file blocking
- Alert generation system

Monitoring Features

- Filesystem event logging
- Transfer monitoring dashboard
- Real-time alert display
- Transfer statistics

Reporting Features

- TXT audit reports
- CSV event reports
- JSON compliance reports
- Visual analytics charts

User Management Features

- User registration and login
- Password hashing
- Role-based access (Admin/User)

DATABASE DESIGN:

The system uses SQLite database with the following tables:

Table Name Purpose

transfers	Stores file transfer details
users	Stores user authentication data
fs_events	Stores filesystem monitoring events

ENCRYPTION AND INTEGRITY MECHANISM:

Encryption Process

1. User uploads a file
2. Password is entered
3. Salt is generated
4. PBKDF2 derives encryption key
5. File is encrypted using Fernet
6. Encrypted data is transferred securely

Integrity Verification

The system generates SHA-256 hashes:

- Before transfer
- After transfer

If both hashes match, the file integrity is verified.

MODULES:

1. Authentication Module

Handles:

- Login
- Registration
- Password verification

2. Dashboard Module

Displays:

- Total transfers
- Alerts
- Sensitive files
- System statistics

3. File Transfer Module

Performs:

- File upload
- Encryption

- Secure transfer
- Integrity validation

4. Alert Module

Monitors suspicious activities and unauthorized actions.

5. Reporting Module

Generates downloadable audit reports and visual analytics.

6. Settings Module

Displays:

- System paths
- Security configurations
- Classification rules

USER INTERFACE:

The project contains the following pages:

Page	Description
Dashboard	Displays monitoring statistics
Send File	Performs secure file transfer
Alerts	Shows suspicious activities
Report	Generates security reports
Settings	Displays system configurations

ADVANTAGES OF THE SYSTEM:

- Secure encrypted file transfer
- User-friendly interface
- Real-time monitoring
- Integrity verification
- Automated reporting
- Lightweight SQLite database
- Easy deployment using Streamlit

LIMITATIONS:

- Works mainly on local systems
- No cloud integration
- Limited multi-user scalability
- Requires manual destination path input

FUTURE ENHANCEMENTS:

- Cloud storage integration
- Email alert notifications
- Malware scanning integration
- Multi-factor authentication
- Real-time filesystem watcher
- AI-based anomaly detection
- Role-based permission management

CONCLUSION:

SECUREXFER v3.0 successfully implements a secure file transfer and monitoring solution using Python and Streamlit. The project ensures confidentiality, integrity, and monitoring of transferred files through encryption, hashing, alert systems, and audit reporting. The system demonstrates practical implementation of cybersecurity concepts such as encryption, integrity validation, access control, and logging in a real-world application.